

Yellow Devil gRPC

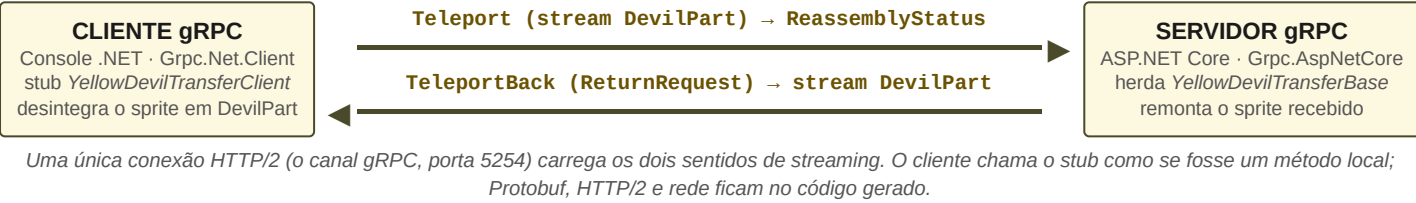
Teletransporte remoto de um chefe de Mega Man (1987) entre processos via gRPC
Disciplina: Desenvolvimento de Sistemas Distribuídos | Ramon Couto Santos & Aaron Goldberg Guerra

Descrição do problema

O sistema simula o **teletransporte do Yellow Devil**, chefe de *Mega Man* (1987), entre **dois processos distintos**. O monstro é famoso por se desmontar em partículas, atravessar a tela e se remontar do outro lado — e é exatamente essa mecânica que vira comunicação remota aqui. O sprite ASCII 52×36 é representado como um conjunto de **1.328 partículas** (pixels com posição X/Y e cor), e **cada partícula é uma mensagem Protocol Buffers**. O cliente desintegra o monstro no seu terminal e transmite as partículas ao servidor por **streaming**; o servidor **reconstrói o desenho partícula a partícula, em tempo real**, no terminal dele. Uma segunda operação permite pedir o monstro de volta, invertendo o sentido do streaming. O objetivo é demonstrar, de forma visual e imediata, como o gRPC transmite dados estruturados entre processos separados a partir de **um único contrato .proto compartilhado**.

CONTRATO
Um **serviço gRPC** (YellowDevilTransfer) com **duas operações remotas** (Teleport e TeleportBack) e a **mensagem composta** DevilPart, de 4 atributos. Contrato completo em YellowDevilServer/Protos/yellow_devil.proto.

Diagrama da comunicação cliente <-> servidor



Mensagem composta e operações

Elemento do contrato	Tipo	Papel
DevilPart	mensagem composta (4 atributos)	part_id (ordem) · pixel_data (cor, em bytes) · target_x e target_y (onde encaixar). A posição viaja junto com o pixel: o receptor não guarda estado para saber onde desenhar.
ReassemblyStatus	mensagem composta (2 atributos)	is_complete + message: confirmação tipada ao fim da ida, em vez de um retorno vazio.
Teleport	client streaming	Operação 1 (ida): as ~1.328 partículas fluem do cliente para o servidor num só stream.
TeleportBack	server streaming	Operação 2 (volta): o cliente pede uma vez e o servidor empurra as partículas de volta. ReturnRequest.shuffle deixa o cliente decidir se voltam embaralhadas.

Capturas de tela do sistema em funcionamento

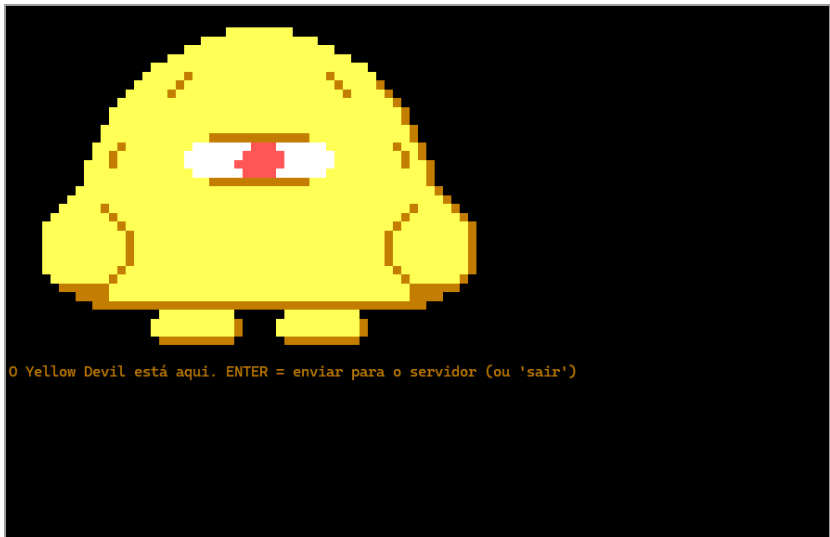


Figura 1 — Yellow Devil montado no terminal do cliente, antes do teleporte. O rodapé mostra o controle: ENTER envia o monstro para o servidor.

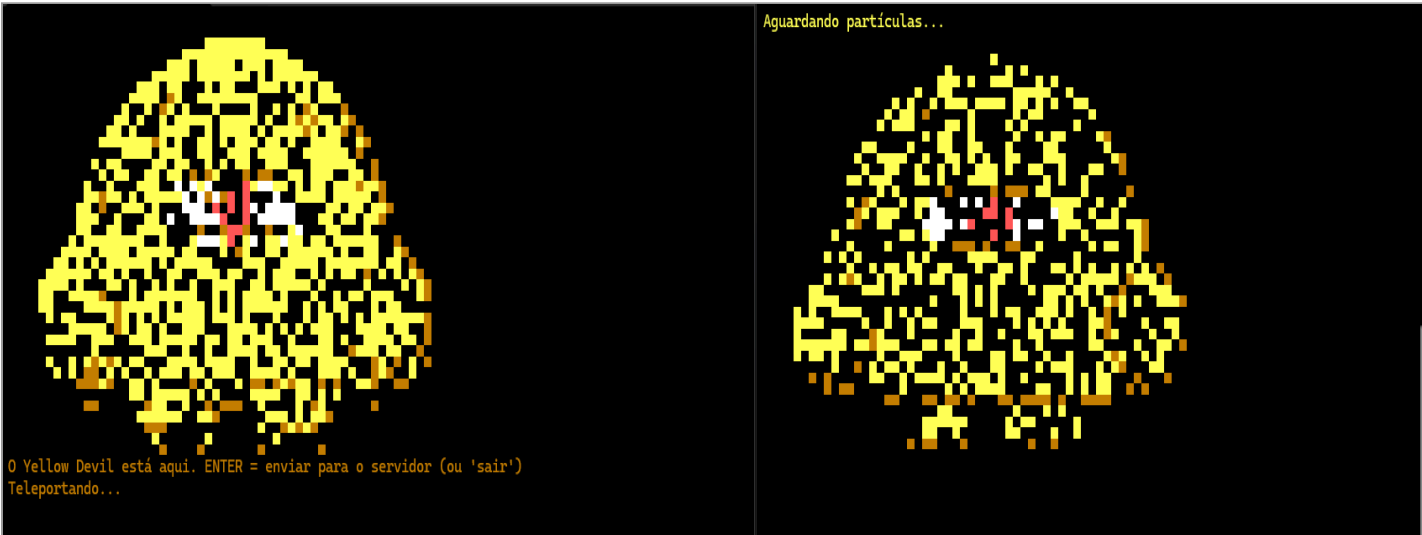


Figura 2 — O teleporte em andamento, com os dois processos lado a lado: à esquerda o cliente (“Teleportando...”) se desintegrando conforme envia as `DevilPart`; à direita o servidor (“Aguardando partículas...”) desenhando cada partícula à medida que chega pelo stream.

Justificativa das decisões de implementação

Usamos **C#** com `Grpc.AspNetCore` e `Grpc.Net.Client` por familiaridade e porque o pacote `Grpc.Tools` **gera os stubs no próprio dotnet build**, sem passo manual. Optamos por **streaming nos dois sentidos** (client streaming na ida, server streaming na volta) porque o problema é naturalmente incremental: cada pixel é uma unidade independente, e **é o stream em andamento que produz a animação** de desmontar e remontar. Uma RPC unária por partícula custaria 1.328 idas e voltas; um único campo `repeated` entregaria o boss num piscar, sem animação. A mensagem `DevilPart` concentra ordem, cor e posição de destino para ser **auto-suficiente**, mantendo o contrato simples. Os dados ficam **em memória** (`List/Dictionary`), pois o escopo dispensa persistência, e o servidor protege o estado com `lock` para que, se dois clientes pedirem o monstro juntos, só o primeiro leve. O **terminal** foi escolhido como interface para eliminar GUI e manter o foco na comunicação remota, que é o objetivo da atividade.